

SysPilot Master Systems

Architecture

Enterprise-Grade Production Linux Observability & Causal
Intelligence Platform

A First-Principles Engineering Design Specification for High-Throughput, Sub-Microsecond
Systems Observability

Production SLA: < 1% CPU | < 64MB RAM | 5M+ Events/sec

Author: Principal Systems Infrastructure & Linux Kernel Engineering Group

Document ID: SIOP-ARCH-SPEC-2026-V4

Target OS: Modern Enterprise Linux (Kernels 5.4+ through 6.x+)

Date: July 2026

1. Executive Summary & Design Principles

Core Architectural Mandate

SysPilot Infrastructure Observability Platform (SIOP) is engineered to capture, enrich, index, correlate, and analyze millions of system telemetry events per second per host with guaranteed host non-interference. It enforces a strict upper resource cap of **< 1.0% host CPU utilization** and **< 64 MiB resident RAM**.

1.1 First-Principles Architectural Axioms

Modern Linux observability platforms often fail at scale due to excessive CPU overhead from high-frequency polling, cache-unfriendly memory layouts, lock contention under core count scaling, and naive dynamic string allocation. SIOP solves these fundamental constraints through five core axioms:

1. **Push over Poll:** Complete elimination of high-frequency filesystem scans (such as periodic `/proc` iteration). All state transitions are event-driven via Linux kernel eBPF tracepoints and multicast Netlink process sockets.
2. **Zero-Copy Kernel Reserve:** Payload allocation occurs inside kernel-space eBPF ring buffers (`BPF_MAP_TYPE_RINGBUF`). Memory pages are directly mapped to userspace, eliminating intermediary buffer copies.
3. **Decoupled Execution Domains:** Heavy symbolization (DWARF unwinding), string formatting, cgroup path resolution, and graph edge logic are completely decoupled from kernel context and executed out-of-band in userspace.
4. **Mechanical Sympathy & L1/L2 Cache Optimization:** Data structures are aligned to 64-byte CPU cache lines. Open-addressing hash maps (`tsl::robin_map`) and lock-free MPMC queues (`moodycamel::ConcurrentQueue`) maximize instruction IPC.
5. **Fail-Safe Self-Preservation:** Adaptive dynamic load shedding throttles telemetry fidelity when host CPU utilization exceeds threshold parameters, guaranteeing system stability.

1.2 System Metric SLA & Verification Matrix

Metric Name	Target SLA	Guaranteed Boundary	Verification Strategy
Host CPU Overhead	< 0.5%	< 1.0% of 1 core	Continuous <code>getrusage()</code> monitoring & cgroup hard quotas
Agent Memory (RSS)	32 MiB	64 MiB Resident	Pre-allocated slab bump arenas (zero dynamic heap growth)
Kernel Latency	< 100 ns	< 500 ns	eBPF probe execution timing via <code>bpf_ktime_get_ns()</code>
Event Ingestion Rate	2,000,000 / sec	5,000,000 / sec / host	SIMD vectorized batch ingestion benchmarks
Context Switches	< 50 / sec	< 100 / sec	Adaptive epoll wakeup timer high-watermark algorithm

2. Kernel Telemetry Subsystem & eBPF

Mechanics

The kernel collection subsystem leverages Linux eBPF (Extended Berkeley Packet Filter) with Compile Once - Run Everywhere (CO-RE) architecture enabled by BPF Type Format (BTF) metadata.

2.1 eBPF Probe Types & Attachment Specifications

Probe Category	Kernel Subsystem	Attach Point / Interface	Telemetry Captured
Tracepoints	Scheduler / Syscalls	tp/sched/sched_switch, tp/syscalls/sys_enter_*	CPU context switches, syscall latency, arguments, return codes
Kretprobes / Fexit	VFS / Network	fexit/vfs_read, fexit/tcp_v4_connect	File read/write byte rates, TCP connection latencies & socket metrics
Uprobes / USDT	User Runtimes	uprobe/libssl.so:SSL_write	Unencrypted TLS payloads & application runtime trace points
Perf Events	Hardware PMU	perf_event_open (PEBS/LBR)	LLC cache misses, branch mispredictions, 99Hz stack traces
Netlink PROC	Kernel Process CN	NETLINK_CONNECTOR (Group CN_IDX_PROC)	Process fork, exec, exit, and uid transition events

2.2 eBPF Event Header Structure (64-byte Cache Line Aligned)

```
// 64-byte cache aligned event structure shared between kernel and userspace
struct __attribute__((packed, aligned(64))) bpftimevent_t {
```

```

uint64_t timestamp_ns; // Kernel monotonic timestamp (bpf_ktime_get_ns)

uint32_t pid; // Process ID

uint32_t tid; // Thread ID

uint32_t ppid; // Parent Process ID

uint32_t uid; // User ID

uint32_t cgroup_id; // Kernel cgroup v2 ID

uint16_t event_type; // System call / tracepoint event code

uint8_t cpu_id; // CPU core ID

uint8_t flags; // Event status flags (0x1 = drop, 0x2 = anomaly)

int64_t ret_val; // Syscall exit code or return value

uint64_t arg1; // Generic payload argument 1 (e.g. fd, address)

uint64_t arg2; // Generic payload argument 2 (e.g. byte length)

uint64_t stack_id; // eBPF stack trace map identifier
};

```

2.3 Kernel eBPF Map Declarations

```

// 1. Shared Global Ring Buffer (Zero-Copy Kernel-Userspace IPC)

```

```

struct {

```

```

__uint(type, BPF_MAP_TYPE_RINGBUF);

__uint(max_entries, 16 * 1024 * 1024); // 16 MiB pre-allocated ring

} events_ringbuf SEC(".maps");

// 2. Kernel Stack Trace Map

struct {

__uint(type, BPF_MAP_TYPE_STACK_TRACE);

__uint(max_entries, 32768);

__uint(key_size, sizeof(uint32_t));

__uint(value_size, 128 * sizeof(uint64_t)); // Up to 128 frame instruction pointers

} stack_traces SEC(".maps");

// 3. Dynamic PID Filter Configuration (Per-CPU Array for lockless reads)

struct {

__uint(type, BPF_MAP_TYPE_PERCPU_ARRAY);

__uint(max_entries, 1);

__type(key, uint32_t);

```

```
__type(value, struct filter_config_t);
```

```
} filter_cfg SEC(".maps");
```

3. Edge Daemon Architecture & Hot-Path Pipeline

The userspace daemon (`syspilotd`) processes raw kernel event streams through a lockless, multi-threaded vector pipeline.

3.1 Zero-Allocation Memory Engine (StringArena)

To eliminate dynamic heap allocations (`malloc/free`) during hot-path processing, SIOP implements a slab bump-allocator memory model:

```
namespace syspilot::memory {  
  
    class StringArena {  
    private:  
  
        static constexpr size_t CHUNK_SIZE = 256 * 1024; // 256 KiB Slabs  
  
        struct Chunk {  
  
            alignas(64) char data[CHUNK_SIZE];  
  
        };  
  
        std::vector<std::unique_ptr<Chunk>> chunks_;
```

```
size_t current_chunk_{0};
```

```
size_t current_offset_{0};
```

```
public:
```

```
StringArena() {
```

```
    chunks_.push_back(std::make_unique<Chunk>());
```

```
}
```

```
std::string_view allocate(std::string_view str) {
```

```
    size_t len = str.size();
```

```
    if (current_offset_ + len > CHUNK_SIZE) {
```

```
        current_chunk_++;
```

```
        if (current_chunk_ >= chunks_.size()) {
```

```
            chunks_.push_back(std::make_unique<Chunk>());
```

```
        }
```

```
        current_offset_ = 0;
```

```
    }
```

```
char* dest = &chunks_[current_chunk_]->data[current_offset_];
```

```
std::memcpy(dest, str.data(), len);
```

```
current_offset_ += len;
```

```
return std::string_view(dest, len);
```

```
}
```

```
void reset() noexcept {
```

```
current_chunk_ = 0;
```

```
current_offset_ = 0;
```

```
}
```

```
};
```

```
} // namespace syspilot::memory
```

3.2 SIMD Vectorized Event Filtering (AVX2 Execution)

```
namespace syspilot::pipeline {
```

```

struct alignas(64) EventBatch {

    static constexpr size_t BATCH_CAPACITY = 1024;

    bpftrace_event_t events[BATCH_CAPACITY];

    size_t count{0};

};

class SIMDFilter {

public:

    // SIMD AVX2 vectorized PID filtering across 8 events per instruction cycle

    size_t filter_pids_avx2(EventBatch& batch, uint32_t target_pid) {

        size_t write_idx = 0;

        __m256i target_vec = _mm256_set1_epi32(static_cast<int>(target_pid));

        for (size_t i = 0; i < batch.count; i += 8) {

            int pids[8] = {

                static_cast<int>(batch.events[i+0].pid), static_cast<int>(batch.events[i+1].pid),

                static_cast<int>(batch.events[i+2].pid), static_cast<int>(batch.events[i+3].pid),

```

```

static_cast<int>(batch.events[i+4].pid), static_cast<int>(batch.events[i+5].pid),

static_cast<int>(batch.events[i+6].pid), static_cast<int>(batch.events[i+7].pid)

};

__m256i loaded_pids = _mm256_loadu_si256(reinterpret_cast<const __m256i*>(pids));

__m256i cmp_mask = _mm256_cmpeq_epi32(loaded_pids, target_vec);

int mask = _mm256_movemask_epi8(cmp_mask);

for (size_t j = 0; j < 8 && (i + j) < batch.count; ++j) {

    if (batch.events[i + j].pid == target_pid || target_pid == 0) {

        batch.events[write_idx++] = batch.events[i + j];

    }

}

}

}

batch.count = write_idx;

return write_idx;

}

};

```

```
} // namespace syspilot::pipeline
```

4. Columnar TSDB Storage & Inverted Indexing

The central analytical engine organizes telemetry into a high-density, columnar Log-Structured Merge-Tree (LSM) format with custom encoding per data type.

4.1 Columnar Encoding Algorithms

Telemetry Attribute	Data Encoding Format	Compression Ratio	Analytical Benefit
Timestamps	Double-Delta Encoding	~10:1	Dense integer delta packing; fast vector scans
Float Metrics	Gorilla / Chimp Floating Point	~6:1	XOR bitwise difference compression for CPU/mem rates
Process / Cgroup Tags	Dictionary Encoding + RLE	~15:1	Integer symbol mapping; minimal storage overhead
Inverted Index Tags	Compressed Roaring Bitmaps	~20:1	Nanosecond set operations (AND, OR, NOT) across attributes

5. Multimodal Causal Reasoning Engine

The causal engine dynamically constructs a topological multigraph (DAG) linking system entities and performs root-cause isolation across correlated telemetry streams.

5.1 Graph Subsystem C++ Interface

```
namespace syspilot::causal {  
  
enum class NodeType : uint8_t { PROCESS, FILE, SOCKET, CGROUP };  
  
enum class EdgeType : uint8_t { SPAWNED_BY, READS_FROM, WRITES_TO, BLOCKED_ON, CONTENTS_WITH };  
  
struct GraphNode {  
    std::string_view id;    // e.g. "pid:4582"  
  
    NodeType    type;  
  
    pid_t    pid;  
  
    double    cpu_pct;  
  
    double    io_rate_kb;  
  
    bool    is_anomalous;  
};
```

```
};

struct GraphEdge {
    std::string_view from_id;

    std::string_view to_id;

    EdgeType    type;

    uint64_t    latency_ns;
};

class CausalGraph {
private:
    tsl::robin_map<std::string_view, GraphNode> nodes_;

    std::vector<GraphEdge>    edges_;

    memory::StringArena    arena_;

public:
    void add_node(GraphNode node) {
```

```
node.id = arena_.allocate(node.id);
```

```
nodes_[node.id] = node;
```

```
}
```

```
void add_edge(GraphEdge edge) {
```

```
    edge.from_id = arena_.allocate(edge.from_id);
```

```
    edge.to_id = arena_.allocate(edge.to_id);
```

```
    edges_.push_back(edge);
```

```
}
```

```
std::vector<std::string_view> trace_root_cause(std::string_view symptom_id) {
```

```
    std::vector<std::string_view> path;
```

```
    tsl::robin_set<std::string_view> visited;
```

```
    std::queue<std::string_view> q;
```

```
    q.push(symptom_id);
```

```
    visited.insert(symptom_id);
```

```
while (!q.empty()) {  
  
    auto curr = q.front();  
  
    q.pop();  
  
    path.push_back(curr);  
  
    for (const auto& edge : edges_) {  
  
        if (edge.from_id == curr && visited.find(edge.to_id) == visited.end()) {  
  
            visited.insert(edge.to_id);  
  
            q.push(edge.to_id);  
  
        }  
    }  
  
    return path;  
}  
};
```

```
} // namespace syspilot::causal
```

6. Production Safety & Reliability Guarantees

The 13 Production Safety Principles

1. **CPU Limit (< 1%):** Push-based eBPF tracepoints + epoll reactor pattern.
2. **Memory Bound (< 64MB):** Pre-allocated slab bump arenas; zero dynamic heap growth.
3. **Zero Hot-Path Heap Allocations:** In-place ring buffer processing and arena strings.
4. **Lockless Operations:** Per-CPU eBPF maps, moodycamel::ConcurrentQueue, atomic markers.
5. **Context Switch Minimization:** Adaptive wakeup high-watermark epoll polling.
6. **Kernel-Userspace Separation:** Raw event emission in eBPF; DWARF unwinding in userspace.
7. **Backpressure Handling:** Atomic event drop counters in kernel eBPF ring buffer headers.
8. **Event Loss Detection:** Monotonic sequence numbers per CPU stream.
9. **Adaptive Dynamic Load Shedding:** Automated token bucket rate limiting at 80% CPU utilization.
10. **Kernel Instability Immunity:** Static verification by Linux kernel eBPF verifier.
11. **Process Fault Isolation:** Independent daemon execution sandboxed in cgroups.

12.**Vectorized Ingestion:** AVX2/AVX-512 SIMD pipeline supporting >5M events/sec.

13.**Resource Cgroup Enforcement:** Enforced systemd slice limits (MemoryMax=128M,
CPUQuota=5%).